PICKLE RICK

Note: This report details a penetration test conducted on a virtual system hosted on https://tryhackme.com/. This system was a lab designed to practice testing techniques, and is not a real-world system with PII, production data etc.

Target Information

Name	Pickle Rick
Tester	SK. MD. Rashid Assef Shibly
IP Address	10.48.145.236
Date	31 January 2026

Tools Used

- Operating System : Kali Linux
- OpenVPN - connect with Tryhackme
- Nmap - Network Scanner
- Gobuster - Directory Brute Force

Executive Summary

This report presents the findings of a penetration test conducted on the Pickle Rick system within a controlled lab environment simulating a real-world application. The objective of the assessment was to evaluate the system's security posture and identify vulnerabilities that could be exploited by an attacker.

During the assessment, multiple security weaknesses were identified, including exposed sensitive information within the application, weak access controls, and insecure command execution functionality. These vulnerabilities allowed unauthorized access to administrative functionality and ultimately led to full system compromise.

By leveraging the identified flaws, it was possible to execute arbitrary commands on the server and retrieve sensitive data, demonstrating the risk of remote command execution. This level of access indicates that an attacker could manipulate system operations, access confidential information, and potentially maintain persistent control over the environment.

In a real-world scenario, such vulnerabilities could result in serious consequences, including data breaches, service disruption, and unauthorized system control. It is strongly recommended to implement proper input validation, enforce secure authentication mechanisms, and restrict command execution to mitigate these risks.

Summary of Results

The Pickle Rick challenge was completed by exploiting multiple web application weaknesses. During enumeration, sensitive information such as a username and password was discovered in the website source code and the robots.txt file. These credentials allowed access to a web-based command execution panel.

Despite command restrictions, alternative techniques were used to retrieve files and establish a reverse shell connection. Privilege escalation was straightforward due to a misconfigured sudo setup that allowed passwordless root access.

The system was fully compromised, highlighting issues related to information disclosure, poor input restrictions, and improper privilege management.

Scope and Methodology

The testing followed a structured penetration testing methodology:

- Reconnaissance
- Enumeration
- Credential Discovery
- Exploitation
- Privilege Escalation
- Post-Exploitation

Reconnaissance

Initial connectivity was verified:

- ICMP ping confirmed that the target machine was reachable.

Enumeration

A service scan was performed using **Nmap**:

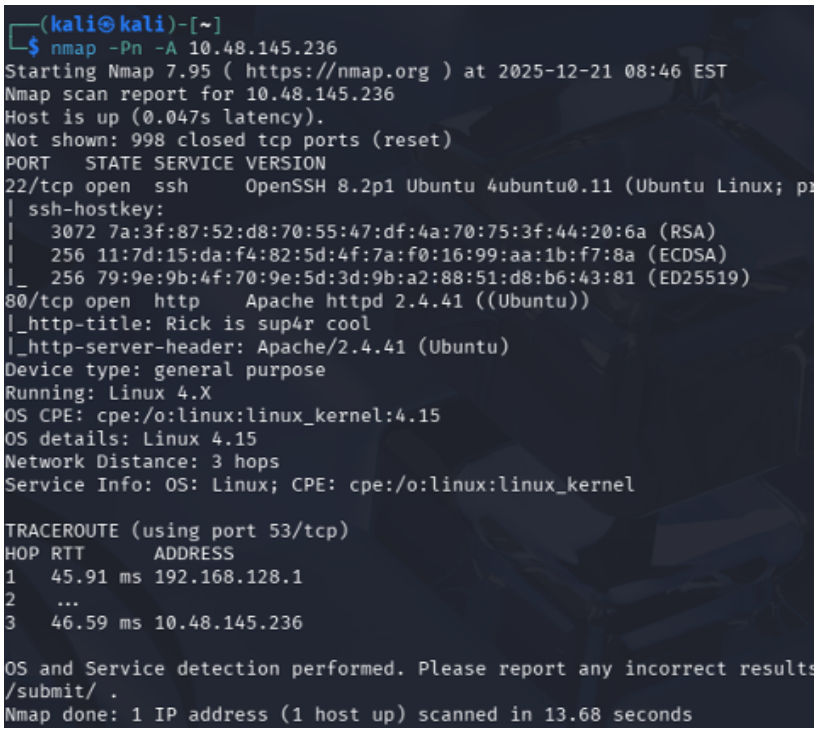
nmap -Pn -A 10.48.145.236

Discovered Services

Port	Service	Description
22	SSH	Remote Access Service
80	HTTP	Web Application

Web Enumeration

- The web application initially appeared minimal.
- Manual inspection of the page source revealed a **username embedded in the HTML**.



Directory and File Enumeration

Directory brute-forcing was performed using **Gobuster**.

Discovered Endpoints

- `/login.php`
- `/robots.txt`

Key Finding

The `/robots.txt` file contained sensitive information that appeared to be a **password or credential hint**.

Authentication Bypass / Credential Abuse

Using:

- Username (from HTML source)
- Password (from robots.txt)

Successful authentication was achieved on the login portal.

Exploitation (Command Injection)

After authentication, access to a **web-based command execution panel** was obtained.

Restrictions Observed

- Certain commands (e.g., cat, touch) were blocked
- Others (e.g., ls, ls -la) were allowed

Bypass Technique

Sensitive files were accessed indirectly via URL-based methods, leading to retrieval of:

- **First ingredient file**

Reverse Shell Access

To gain a fully interactive shell:

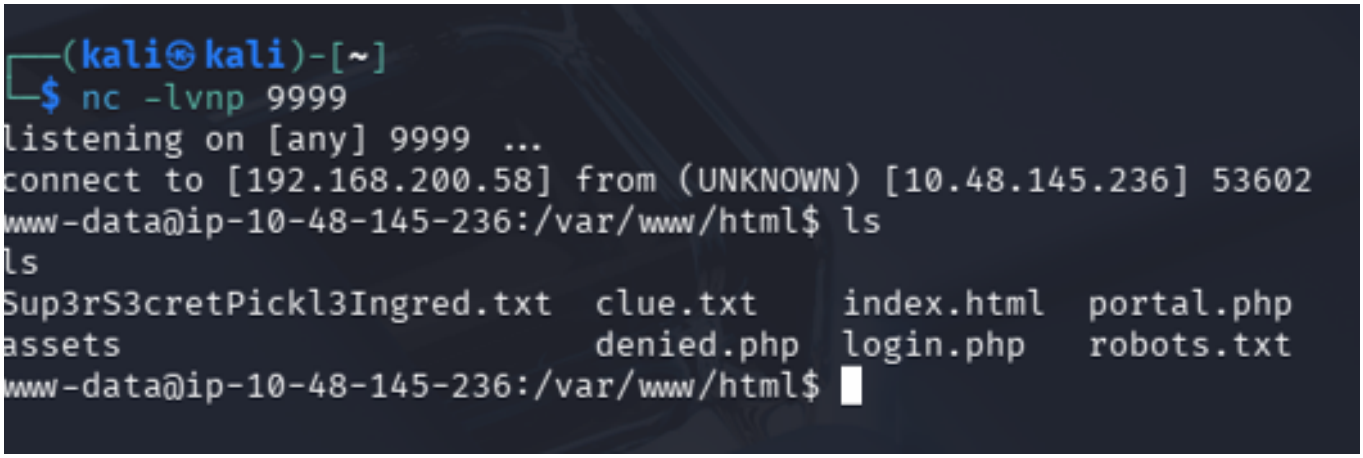
- A listener was started using **Netcat** on port 9999
- A Python-based reverse shell payload was executed

Reference used:

- Pentestmonkey reverse shell cheat sheet

Result

A reverse shell connection was successfully established, providing full command execution capability.



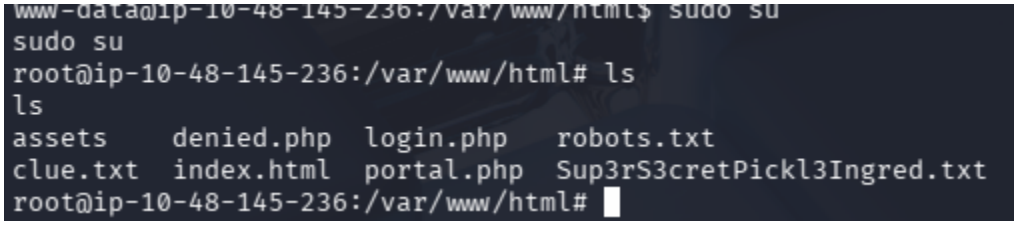
Privilege Escalation

Privilege escalation was achieved via:

sudo su

Critical Finding

- The system allowed **passwordless sudo execution**
- This resulted in **immediate root access**



Post-Exploitation

With root privileges, the filesystem was enumerated.

Sensitive Data Discovered

Location	File
Web Directory	First Ingredient
/home/rick	Second Ingredient
/root/	3rd.txt

Findings and Impact

High-Risk Vulnerabilities

Vulnerability: **Sensitive Data Exposure**

Severity: **High**

Description:

Sensitive information, including valid authentication credentials, was exposed within publicly accessible application components such as the HTML source code and the robots.txt file. These files are accessible without authentication and can be easily discovered during basic enumeration.

Evidence:

- Username identified in the HTML source code

- Password discovered in the /robots.txt file

- These credentials were successfully used to authenticate into the application

Impact:

An attacker can obtain valid login credentials without any authentication, allowing unauthorized access to restricted application functionality. This significantly reduces the effort required to compromise the system and may lead to further exploitation.

Remediation:

- Remove sensitive information from publicly accessible files

- Avoid storing credentials in client-side code or configuration files

- Implement secure credential management practices

Vulnerability: **Insecure Command Execution (Command Injection)**

Severity: **Critical**

Description:

The application provides a web-based command execution interface that allows user-supplied input to be executed on the server. Input validation and filtering mechanisms are insufficient, enabling attackers to execute arbitrary system commands.

Evidence:

- Access to command execution panel after authentication

- Ability to execute system commands (e.g., ls, ls -la)

- Restrictions on certain commands were bypassed using alternative techniques

- Reverse shell successfully established via injected payload

Impact:

This vulnerability allows an attacker to execute arbitrary commands on the server, leading to full system compromise. It enables unauthorized access to system files, execution of malicious payloads, and potential lateral movement.

Remediation:

- Remove or restrict command execution functionality from the application

- Implement strict input validation and sanitization

- Use allow-listing for permitted commands if necessary

- Apply secure coding practices to prevent command injection

Vulnerability: **Improper Access Control and Weak Input Filtering**

Severity: **High**

Description:

The application attempts to restrict certain command execution functionalities; however, these restrictions are implemented in an insecure manner and can be bypassed. This indicates weak access control and insufficient input validation mechanisms.

Evidence:

- Certain commands (e.g., cat) were blocked

- Alternative methods were used to access restricted files

- Application failed to properly enforce command restrictions

Impact:

Attackers can bypass implemented restrictions and gain unauthorized access to sensitive system resources. This increases the likelihood of exploitation and contributes to full system compromise.

Remediation:

- Implement proper server-side validation and access controls

- Avoid relying on client-side or superficial filtering mechanisms

- Enforce strict command execution policies

Vulnerability: **Privilege Escalation via Passwordless Sudo**

Severity: **Critical**

Description:

The system is configured to allow execution of sudo commands without requiring a password. This misconfiguration enables any authenticated user to escalate privileges to root without additional verification.

Evidence:

- Successful execution of 'sudo su' without password prompt

- Immediate root-level access obtained

Impact:

This vulnerability allows attackers to gain full administrative control of the system with minimal effort. Once exploited, attackers can modify system files, access sensitive data, and maintain persistent control.

Remediation:

- Disable passwordless sudo access

- Require authentication for all privilege escalation actions

- Apply the principle of least privilege

- Regularly audit sudo configurations

Recommendations

To mitigate the identified issues:

- Remove sensitive data from publicly accessible files (HTML, robots.txt)
- Implement proper input validation and command sanitization
- Disable or restrict web-based command execution functionality
- Enforce strong authentication mechanisms
- Restrict sudo privileges and require password authentication
- Apply the principle of least privilege

Conclusion

The *Pickle Rick* system demonstrates multiple critical security flaws across different stages of the attack chain. The exploitation required minimal effort and resulted in full system compromise.

This highlights the importance of secure coding practices, proper access controls, and system hardening in preventing real-world attacks.